Ideation Phase
Brainstorm & Idea Prioritization Template

Date	**27.03.2026**
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

Brainstorm & Idea Prioritization Template:

- # Phase 1: Brainstorming & Requirements

Before touching the instance, define the "Who, When, and Why." Use this table to map out your logic.

Field Name	Table	Condition (When should it be mandatory?)	Reverse if False?
`Work Notes`	Incident	When State is "Resolved" or "Closed"	Yes
`Resolution Code`	Incident	When State changes to "Resolved"	Yes
`Assignment Group`	Change Request	If "Urgency" is High	No

Key Questions to Ask:

- **Data Integrity:** Will making this mandatory break existing integrations or automated scripts?
- **User Experience:** Are we overwhelming the user with too many red asterisks at once?
- **Consistency:** Should this be a **Data Policy** instead? (UI Policies only work on the browser; Data Policies enforce rules at the server level/import sets).

2. Phase 2: Prioritization Matrix

Not every "mandatory" request is urgent. Prioritize based on **Data Value** vs. **User Friction**.

- **P1: Regulatory/Compliance:** Fields required for auditing (e.g., Change Request "Reason for Change").
- **P2: Operational Necessity:** Fields required for routing (e.g., "Assignment Group").
- **P3: Nice-to-Have:** Fields for reporting enrichment (e.g., "Source of Call").

3. Phase 3: Implementation Strategy (UI Policies)

When building the UI Policy, follow these best practices to ensure they are "Update Set friendly."

Step-by-Step Checklist:

- **Check for Conflict:** Verify no Client Scripts are already manipulating these fields.
- **Define the Policy:**
  - **Short Description:** Use a naming convention like `[Table] - Make [Field] Mandatory on [Condition]`.
  - **Global:** Ensure it's checked if it applies to all views.
  - **OnLoad:** Usually checked to evaluate the condition as soon as the record opens.
- **UI Policy Actions:**
  - Create a related record for each field.
  - Set **Mandatory** to **True**.
  - **Tip:** Avoid "Leave Alone" if you want predictable behavior.

4. Phase 4: Migration via Update Sets

This is where most "dirty" data enters production. Follow this flow to keep your migration clean.

The "Clean Migration" Workflow:

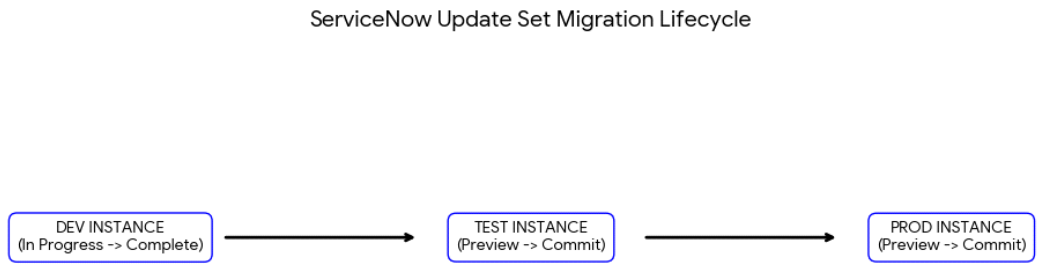
- **Open Update Set:** Name it clearly: `USER_TASK_ShortDescription_V1`.
- **Capturing:** Only perform actions related to the UI Policy. Avoid "window shopping" (clicking into other records) to prevent accidental captures.
- **Validation:**
  - Navigate to **Local Update Sets**.
  - Review the **Customer Updates** list. You should see entries for `sys_ui_policy` and `sys_ui_policy_action`.
- **Closing:** Set to **Complete**.
- **Deployment:**
  - **Preview** on the target instance (Test/Prod).
  - **Check for Errors:** Resolve any "Missing Record" or "Local Version Newer" errors before committing.

5. Quick Reference Template

Copy and paste this into your project documentation:

Objective: Enforce [Field Name] as mandatory.

Logic: If [Condition] is met, then [Field] = Mandatory.

Update Set Name: `Initial_UI_Policy_Enforcement`

Rollback Plan: Set UI Policy to `Active = false` and migrate in a new Update Set if issues arise in UAT.